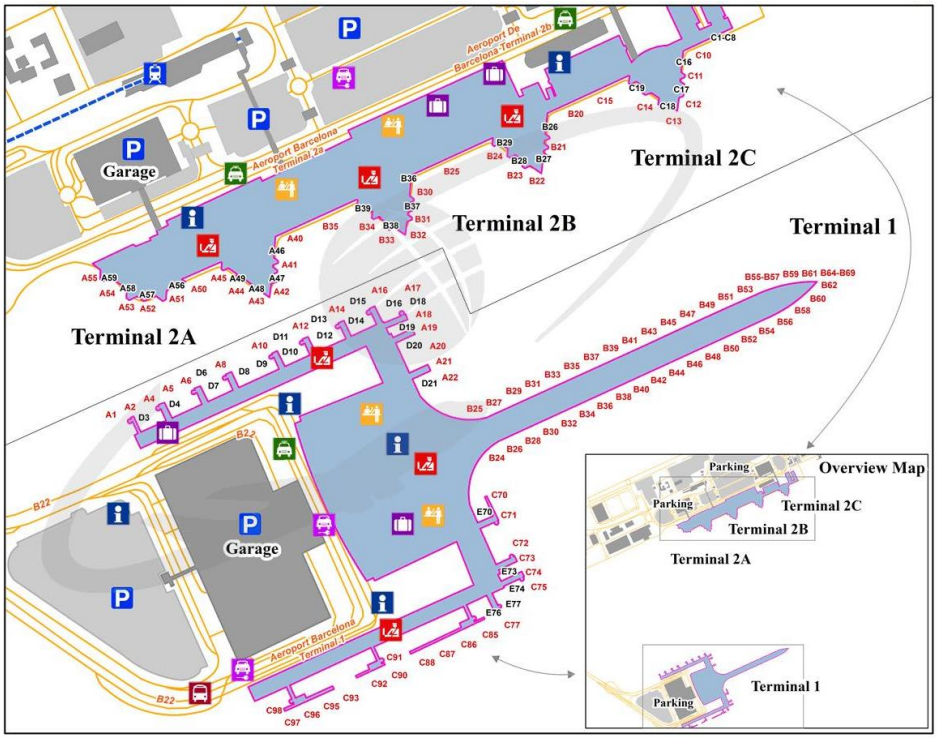
Informàtica 1

Semester 2025-26 Q2

Airport Management

Project statement



Introduction

The project consists of the development of a tool that facilitates the management of the gates in an airport. The airport can have several terminals. Each terminal can have several boarding areas and each of these areas has several gates. The tool must be able to assign gates to the flight that is going to arrive at the airport (keeping in mind that the flights of a certain company always go to the same terminal). It should also display graphs that show different occupancy data. The tool must have a friendly interface that facilitates the user to carry out all these operations.

During the process you must deliver four versions of the tool, of increasing complexity. This document describes the requirements for each version. The first version will be done in a very guided way, step by step. The following versions will not be so guided, so that you can train your abilities to plan work, distribute tasks, integrate partial results, etc. The latest version has few mandatory requirements because we hope that in that version you will put your initiative and creativity into play to incorporate functionalities into the tool that surprise us.

The description of each version includes quality criteria that teachers will use to determine what has been done well and what has not been done well in that version. These criteria should be a useful guide for you to ensure that you are developing the project well. Take into account that quality criteria are incremental. That means that que quality criteria for a given version include all the quality criteria for previous versions and additional quality criteria for that new version.

Each of the versions will be evaluated by teachers, using the corresponding quality criteria. The evaluation of the first three versions will be purely formative. That is, the teachers will not give any grade, but they will give you comments that should help you improve for the next version. Only the latest version will be graded (the quality criteria for that version will indicate how it will be graded). Remember that this grade will be the same for all members of the group and that it has a weight of 30% in the course grade.

In the subject planning that appears in Atenea, the delivery date of each of the versions is clearly indicated. These delivery dates are immutable. There is no excuse for not delivering the corresponding version on time because since it does not have an associated grade, what is not operational in one version is pending for the next. The important thing is that teachers can evaluate each version when planned and can make relevant comments to each team as soon as possible.

Each delivery consists of the developed code and an explanatory video. The code must be delivered in the form of a repository on GitHub, which is a free tool that allows code storage in the cloud and management of versions (exactly what you need for this project). In Atenea it will only be necessary to provide the URL of the GitHub repository that contains your code, which naturally must be publicly accessible.

The video must show in no more than 5 minutes that the tool meets the corresponding requirements. It should also show the parts of code that were added to the previous version, with relevant explanations, including the actions taken by the team as a result of the feedback given by teachers for the previous version. For example, it will be positively valued if external help (from other colleagues or from AI tools such as Copilot or ChatGPT) to incorporate some parts of code has been clearly explained in the video. The link to the video must be included in the README of the project repository in GitHub.

Version 1

The first version of the tool will only deal with the management of information about airports. Each airport has an ICAO code and a geographic position (lat, lon). It is also important to know if the airport is in a country belonging to the Schengen zone.

This first version will have basic functionalities to load a list of airports from a file, add/delete airports, show some graphics and show the location of each airport in Google Earth.

The application must have a friendly graphical interface that allows the user to perform all these operations.

The work is organized in 6 steps described below. It is important to read the instructions and observations of each step carefully.

It is strongly recommended that to develop this first version you work together, side by side, which will facilitate shared learning of those topics that you have studied separately. It may be appropriate to divide a small part of the work among the group members (especially in the last steps). But the time will come, in future versions, for you to test your ability to separate tasks, distribute them equitably and then harmoniously integrate the results.

Step 1

Create the file *airport.py* with the definition of the *Airport* class. The class must have the following attributes and the corresponding constructor (only with code and coordinates).

- ICAO code of the airport (4 characters, ie. LEBL for Barcelona El Prat)
- coordinates (2 real numbers containing airport latitude and longitude in degrees, ie. 41.297445 and 2.0832941 for LEBL)
- Schengen (1 boolean that is True if the airport is in a country belonging to Schengen)

Add to *airport.py* the functions whose headers are the following:

```
IsSchengenAirport (code)
'''Receives the ICAO code of an airport and checks if the airport is in an
Schengen country. The countries which signed the Schengen agreement are
Austria, Belgium, Czech Republic, Cyprus, Denmark, Estonia, Finland, France,
Germany, Greece, Netherlands, Hungary, Iceland, Italy, Latvia, Lithuania,
Luxemburg, Malta, Norway, Poland, Portugal, Slovakia, Slovenia, Spain, Sweden
and Switzerland. To know if an airport belongs to any of these countries you
have to check if the first 2 characters of the ICAO code are one of the
following codes which correspond to the above list of countries:
'LO', 'EB', 'LK', 'LC', 'EK', 'EE', 'EF', 'LF', 'ED', 'LG', 'EH', 'LH',
'BI','LI', 'EV', 'EY', 'EL', 'LM', 'EN', 'EP', 'LP', 'LZ', 'LJ', 'LE', 'ES',
'LS'
The function returns a boolean that is True if the airport belongs to a
Schengen country, and False otherwise. If the input parameter is empty then
False is returned.
'''

SetSchengen (airport)
'''Receives an airport and sets the Schengen attribute (True or False). Uses
the IsSchengenAirport function
'''
```

```
PrintAirport (airport)
#Prints in console the data of the airport
```

Step 2

Create a file *test_airport.py* the with following code:

```
from airport include *
airport = Airport ("LEBL", 41.297445, 2.0832941)
SetSchengen(airport)
PrintAirport (airport)
```

This code will allow you to check if the definitions in *airport.py* are ok. Both files must be in the same folder. You only must run the *test_airport.py* code and check if the results are correct. Try with different airports (including no Schengen).

Step 3

Information from airports is stored in a text file[1]. We need functions to load/save info in/to the file. Add the functions with the following headers to *airport.py*:

```
LoadAirports (filename)
'''Opens the file with name received as input and with the format described
below, with a header line and an airport per line, and returns a list of
airports with the data found in the file. Convert the format of latitude and
longitude from string to decimal degrees (float that will be negative for West
and South coordinates). In the file you will not find any data about Schengen,
so do not update this field of the structure. If the file does not exist the
function returns an empty list.

CODE LAT LON
BIKF N635906 W0223620
CYUL N452805 W0734429
CYYZ N434038 W0793750
DAAG N364138 E0031252
DTTA N365104 E0101337
EBAW N511122 E0042737
'''

SaveSchengenAirports (airports, filename)
'''Given a list of airports, this function writes the information of the
airports that are Schengen into a file whose name is in filename. The format
of the output file must be the same as the format of the input file described
above. If the vector is empty no file is created and an error code is
returned.
'''
AddAirport (airports, airport)
'''Adds the airport to the list of airports if the airport is not in
the list.
'''
RemoveAirport (airports, code)
''' Remove from the list of airports the airport whose code is
received as parameter. If the airport is not in the list an error code
is returned.
'''
```

1 Find a large file with many airports in ATENEA week 4, named as Airports.txt

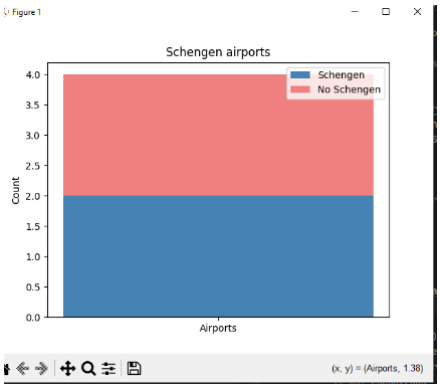
Step 4

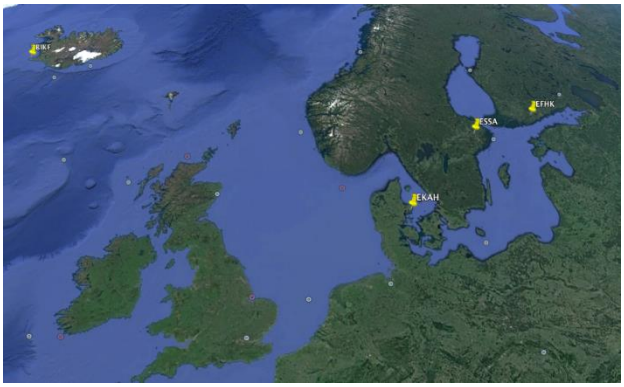
Include in *test_airports.py* the code needed to check that the new functions work well.

Step 5

Let's include some graphics that might make the application more user-friendly. Add to *airport.py* two new functions with the headers shown bellow and include in *test_airports.py* the code needed to check that the new functions work well.

```
PlotAirports (airports)
'''Shows a plot of type stacked bar with
Schengen and non-Schengen airports (see the
figure).
'''
```






```
MapAirports (airports)
'''Shows in Google Earth the
airports in the list. Use different
colors to show Schengen airports
and no Schengen airports. See Annex
A to learn how to show info in
Google Earth.
'''
```

Step 6

Create the *interface.py* file a graphical interface in tkinter that allows you to perform, in the most user-friendly way possible, all the operations implemented in *airports.py*.

Quality criteria for version 1

Correct

- All code files have been developed (*airport.py, test_airport.py* and *interface.py*).
- The script *test_airport.py* allows you to verify the correct functioning of all the functions associated with the Airport class.
- Through the graphical interface the user can:
  - o Load airports
  - o Add/delete airports
  - o Set Schengen attribute to airports
  - o Show data of airports in the list
  - o Save Schengen airports in file
  - o Plot Schengen/No Schengen airports in a stacked bar
  - o Show airports in Google Earth

Video • The video makes it clear which functionalities are operational among those planned for this version. • The video shows the most relevant pieces of code.
Robust • The application resists without blocking the errors that the user may make when interacting with it. It never crashes. • The application detects and reports errors when reading files.
Friendly • The user has no doubt when interacting with the graphical interface. • The graphical interface is visually attractive.
Organized and documented • The code is well structured and indented. It is easy to locate where in the code every functionality is implemented. • Clarifying comments have been added. • The github repo is well organized: includes the developed code and a Readme with the name (and a picture) of the team members, the link to the video and a tag for the version.

Version 2

This version we will include information about flights arriving to Barcelona El Prat airport (LEBL). For each of these flights we want to know the id, the company, the origin airport, and the expected time for landing. We will include in the application functionalities to load flight information from files and show different types of plots.

The development of version is much less guided than the text for version 1. Read carefully all the requirements that will be described in the following paragraphs and consider the distribution of tasks between team members.

You will need to create a file aircraft.py with the definition of class Aircraft, with the following attributes:

- aircraft id (a string)
- airline company (3 characters with the ICAO code of the airline)
- origin airport (4 characters with the ICAO code of the airport the aircraft is coming from)
- time of landing in Barcelona (5 characters with format hh:mm)

Include in *aircraft.py* the functions with the following headings:

```
def LoadArrivals (filename)
''' Opens the file with name received as input and with the format described
below, and returns a list of aircraft initialized with the data found in the
file. These are the flights that will arrive to LEBL at a given day. In the
file you will not find all data defined in the structure Aircraft, so update
only the fields of the structure you can. If the file does not exist the
function returns an empty list. In case some of the aircraft lines do not have
a correct time or the expected structure, then the function must skip this
line and proceed with the rest of lines in the file. Note that the arrivals
file is sorted by landing time.

AIRCRAFT ORIGIN ARRIVAL AIRLINE[2]
ECMKV LYBE 0:04 VLG
ECJGM EGCC 0:05 VLG
ECLOB LMML 1:24 VLG
ECLVC LGTS 2:28 VLG
DALEC EBBR 3:14 BCS
N327UP EDDK 3:17 UPS
'''
```

```
def PlotArrivals (aircrafts)
''' This function receives a list of aircraft and shows a plot of the landing
frequency during the day, this is the number of landing aircraft in every hour
period. If the list is empty an error message shall be displayed in the screen
and return.
'''
```

```
def SaveFlights(aircrafts, filename)
''' Writes the information in the list of aircrafts into a file. The output
format mut be the same as the input format shown above. Be aware that some
parts of the Aircraft structure can be empty field values. Write them as a
string with '-' or as number 0. If the vector is empty no file is created and
an error core is returned.
'''
```

```
def PlotAirlines (aircrafts)
''' Receives a list of aircrafts and shows a bar plot of the number of flights
per airline. If the vector is empty an error message shall be displayed in the
screen and no plot is shown.
'''
```

```
def PlotFlightsType (aircrafts)
''' Receives a list of aircraft and shows a stacked bar plot of the number of
flights arriving from Schengen countries and the number of non-Schengen
(similar to the plot shown by PlotAirports function from version 1). If the
list is empty an error message shall be displayed in the screen and no plot is
shown.
'''
```

```
def MapFlights(aircrafts)
'''Shows in Google Earth the trajectories of all flights in the list, from
origin airport to LEBL. Show in different colors the trajectories with origin
in a Schengen country. Remember that Annex A explains how to draw lines in
Google Earth.
'''
```

2 Find in ATENEA, week 4, a large file named Arrivals.txt with this format

```
def LongDistanceArrivals(aircrafts)
'''Returns a list with the aircrafts from the input list of aircrafts that
arrive to LEBL from an airport that is more that 2000 Km away (this aircraft
would need special inspection after landing).
You will need a function to compute the Haversine[3] distance between two
coordinates.
'''
```

You will need also some code to test all the functions included in *aircraft.py*. But in this case, we will follow a different approach that in the case of *airport.py*. Add at the end of *aircraft.py* the following code:

```
# test section
if __name__ == "__main__":
       aircrafts = LoadArrivals ("arrivals.txt")
       PlotArrivals (aircrafts)
```

This code can be used to test two of the functions. You just need to run the *aircraft.py* script and check the result. The system will detect that you are running *aircraft.py* as a main program and will execute the code in this test section. Of course, you must prepare a file named *arrivals.txt* with information in the format described above. You will find example files in Atenea. So, using this approach you do not need to prepare a separate script to test the definitions in *aircraft.py.* Add to the test section the code needed to test the rest of functions.

Finally, you must extend the user interface with the elements that should allow the user to use the new functionalities.

Additional quality criteria for version 2

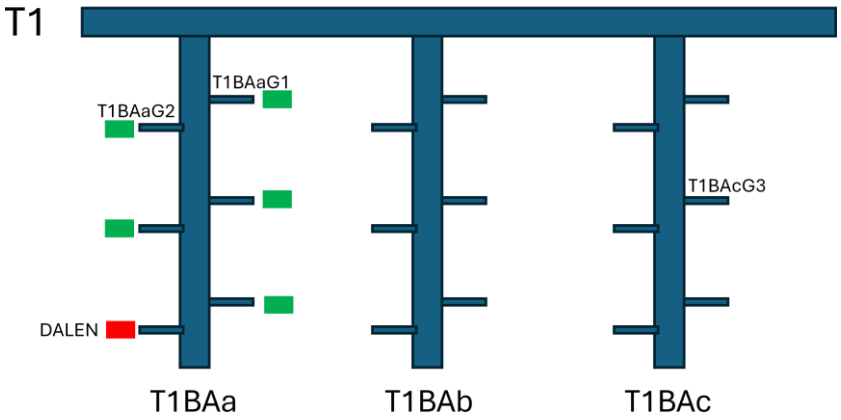
Correct

- File aircraft.py has been included, containing a test section that allows to test all the new functionalities.
- Through the graphical interface the user can:
  - o Load arrivals, selecting the input file
  - o Save aircraft information, selecting the output file
  - o Plot arrivals in every hour period
  - o Plot arrivals per company
  - o Plot Schengen arrivals versus non-Schengen (stacked bar)
  - o Show trajectories in Google Earth
  - o Show only long-distance trajectories in Google Earth

3 See in Annex 2 the basis of the Haversine formula.

Version 3

This version will include new functionalities for gate management in the LEBL airport. The figure is a simple model for airport gate organization. An airport has one or several terminals. All flights of a given company operate always in the same terminal. Every terminal has a name (T1 in the figure) and a list of Boarding Areas (3 in the case of T1). Every Boarding Area has a name (T1BAa, T1BAb and T1BAc in the figure) and a list of gates. Some Boarding Areas are dedicated to flights arriving from Schengen countries and others for non-Schengen countries. Every gate has a name (for instance T1BAaG1) and may be free or occupied by an aircraft (aircraft with id DALEN is in gate T1BAaG6 in the figure).



The information describing the structure of an airport is contained in a text file. For instance, in the case of LEBL, the name of the file is LEBL.txt[4] and the contents is:

```
LEBL  2 terminals
Terminal     T1      5    boarding areas
Area A Schengen      Gates 1 - 11
Area B Schengen      Gates 1 - 57
Area C Schengen      Gates 1 - 11
Area D non-Schengen Gates 1 - 11
Area E non-Schengen Gates 1 - 11
Terminal     T2      6    boarding areas
Area M Schengen      Gates 1 - 8
Area R Schengen      Gates 9 - 19
Area S Schengen      Gates 20 - 30
Area U Schengen      Gates 30 - 39
Area W non-Schengen Gates 40 - 49
Area Y non-Schengen Gates 50 - 59
```

The first line has the ICAO code of the airport and the number of terminals. Then for each terminal there is a line with the name of the terminal and the number of boarding areas of this terminal. Following this terminal header line, there are as many lines as boarding areas, each line describing a boarding area. The lines describing the boarding areas have the name of the area, a description of the type of arrival/departures

4 Find this file in ATENEA, week 4.

(Schengen or non-Schengen) and the initial and final gate numbers found of this boarding area.

Additionally, you have 2 more files: T1_Airlines.txt and T2_Airlines.txt[5]. They both have a list of airlines, on separated lines and sorted in alphabetic order. An airline shall appear only in one of the files, and indicates the terminal in which the airlines have to board their passengers. This in an example of contents for one of these text files. Every line contains the name of the company and the ICAO code separated by a tab.

```
Adria Airways     ADR
Aegean Airlines   AEE
Aeroflot    AFL
Aerolineas Argentinas   ARG
Air Algerie       DAH
Air Baltic        BTI
Air Berlin        BER
```

In order to represent an airport in your application you will need to define a few new classes and functions. Create a file LEBL.py and define the following classes (with the corresponding constructors) related with the airport boarding gates:

- **BarcelonaAP**
- **Terminal**
- **BordingArea**
- **Gate**

Class **BarcelonaAP** will contain a code and a list of objects of class Terminal.

Class **Terminal** will contain a name, a list of objects of class BoardingArea and a list with the ICAO codes of the airline companies which work in the terminal.

Class **BoardingArea** will contain a name, the type (Schengen/nonSchengen) and a list of objects of class Gate.

Class **Gate** will contain a name, a boolean about gate occupancy, and the aircraft id in case the gate is occupied.

Add to LEBL.py the functions whose headers are described in the following. Include also a test section to check if all the functions work well.

```
def SetGates (area, init_gate, end_gate, prefix)
''' Given   area, of class BoardingArea, two numerical values containing first
and last gate number of this boarding area and a string with a prefix for the
gate name, the function updates the list of gates of the boarding area,
assigning to every gate a name obtained by concatenating the prefix and the
gate number, and marking the gate as free. If the area had a previous list of
gates, this will be dropped. If the end_gate number is not greater than the
init_gate number then a code error (-1) is returned.
'''
```

5 Find these 2 files also in ATENEA, week 4.

```
def LoadAirlines (terminal, t_name)

''' Given terminal of class Terminal and t_name containing the name of the terminal, this function reads the information from the file named as {terminal_name}_Airlines.txt and updates the list of airlines of the terminal using the information obtained from the file. If the terminal had a previous list of airlines, this list will be dropped. If the file does not exist, an error code shall be returned and no modification of the terminal parameter shall be done. The format for ***_Airlines.txt files has been described above.

'''

def LoadAirportStructure (filename)

''' Returns an object of class BarcelonaAP built with the information contained in the text file whose name is filename (for instance, LEBL.txt described above). This function shall call the SetGates and LoadAirlines functions defined above in order to complete the structure of the gates in the airport. When calling SetGates use different prefixes for each boarding area to easily locate a gate by its prefix. If the file does not exist am error code must be returned.

'''

def GateOccupancy (bcn)

''' Given bcn of class BarcelonaAP, this function returns a list of gates with their names, their status (free of occupied) and the id of the aircraft in case of occupied.

INTERESTING ADDITION: Use the returning list, with gate occupancy, to call a new function to build a plot showing the airport terminals, boarding areas and gates, with the state of each gate. The figure in page 10 can be a simple example of the type of plot that would be very appreciated by the user. This would be considered as a nice extra functionality (see grading criteria for final version).

'''

def IsAirlineInTerminal (terminal, name)

''' Given  terminal of class Terminal and the name of one airline, this function returns True if the airline is in the list of airlines boarding in this terminal and False otherwise. If the terminal has an empty list of airlines, the return value shall be also False. If the name of the airline is a null string then False and an error code must be returned.

'''

def SearchTerminal (bcn, name)

''' Given bcn of class BarcelonaAP and the name of one airline, this function returns the name of the terminal where the airline must board its passengers. Use function IsAirlineInTerminal. If the airline is not found in any of the terminals, the return name shall be a null string.

'''

def AssignGate (bcn, aircraft)

''' Given bcn of class BarcelonaAP and an aircraft of class Aircraft this function looks for the first gate that is not occupied in the correct boarding area. To decide the correct boarding area the function must check the airline-terminal assignment (using the SearchTerminal function defined above) and the Schengen/non-Schengen type of flight-boarding area. The gate assignment consists in updating the occupancy boolean and the aircraft field of the chosen gate inside the bcn parameter. If there is not more free gates of the correct type, an error code shall be returned and no modification of the bcn parameter shall be done.

'''
```

Finally, you must extend the user interface with the elements that should allow the user to use the new functionalities.

Additional quality criteria for version 3

Correct

- All new classes an functions have been included in file *LEBL.py.*
- The test section that allows you to verify the correct functioning of all the functions associated with the classes.
- Through the graphical interface the user can:
  - o Build the data structure of the LEBL airport
  - o Assign gates to arriving flights
  - o Show gate occupancy (ideally in the form of plot with the airport structure)

Version 4

The two new features of version 4 (final version) are:

- The application must take into consideration aircraft departures (not only arrivals), since departures may free gates that would be available for new arriving aircrafts.
- The application must incorporate the extra functionalities that the team wishes to develop, depending on their motivation and availability.

An arriving aircraft may depart to a new destination some time later (releasing the gate). So, your application must also consider departing flights when assigning gates. Departures must be matched with the arrivals through the aircraft identification. They can also be aircrafts that have spent the night at the airport (there is not corresponding arrival during the day). At the moment of departure, they free the gate and this gate can be assigned to an aircraft arriving after this departure.

The Aircraft structure will be redefined together with its __init__ function to hold a destination airport. Then follows the list of functions that shall be developed. Include also in the test section the code required to test the new functions.

Edit the file aircraft.py and enlarge class Aircraft with the following fields related to departures:

- ...
- destination airport (4 characters with the ICAO code of the airport) the aircraft will depart to after its stay in Barcelona)
- time of departure from Barcelona (5 characters with format hh:mm)

Update the constructor to initialize class Aircraft with the new fields. Then write the following new function related with the new Aircraft.

```
def LoadDepartures (filename)
''' This function opens the file with name received as input, that contains
the information about departures, and returns a list of aircraft initialized
```

```
with the data found in the file. In the file you will not find all data
defined in the structure Aircraft, so update only the fields of the structure
related to departures. If the file does not exist the function returns an
empty list and an error code. Data for departures in the file is in the format
described by this example:

AIRCRAFT DESTINATION DEPARTURE AIRLINE[6]
ECMKV LYBE 0:04 VLG
ECJGM EGCC 0:05 VLG
ECLOB LMML 1:24 VLG
ECLVC LGTS 2:28 VLG
DALEC EBBR 3:14 BCS
N327UP EDDK 3:17 UPS
'''
```

```
def MergeMovements (arrivals, departures)
''' This function receives two lists of aircraft (arrivals and departures) and
returns a new list of aircrafts in which the data of the aircraft with the
same id and with compatible times are merged into the same Aircraft structure.
Times are compatible when the arrival time is previous to the departure time.
Remember that some aircrafts may not have a departure and others may not have
an arrival in the day (night aircraft). If some of the input lists is empty an
error code shall be returned. IMPORTANT: an aircraft can land and take-off
more than once at LEBL during the same day.

'''
```

```
def NightAircraft (aircrafts)
''' This function receives a list of aircraft and returns a new list with the
aircrafts that have not an arrival but only the departure information set. If
the input list is empty an error code must be returned.
'''
```

Add new functions that also relate the aircraft lists with the LEBL airport structure:

```
def AssignNightGates (bcn, aircrafts)
''' This function receives an object of class BarcelonaAP, bcn, and a list of
aircraft. It assigns a gate to each of the aircraft in the list. Use the new
version of function AssignGate. Check that the aircrafts in the list are only
departure flights, with empty data related to arrival. In case an aircraft
does not meet the condition then skip to the following aircraft in list. If
the input list is empty return an error code.
'''
```

```
def FreeGate (bcn, id)
''' This function receives an object of class BarcelonaAP, bcn, and an id of
an aircraft. Updates the bcn object setting to free the status of the gate
assigned to the aircraft. In case the aircraft is not found in any gate an
error core must be returned.
'''
```

When considering both arrivals and departures, the process of gate assignment is a little bit more complicated. In version 3 it has been assumed that once landed the aircraft remains in the assigned gate for the rest of the day. Therefore, the assignment of gates can be done only once at the beginning of the day and that assignment remains static throughout the day.

Imagine now that aircraft A is assigned gate G, lands at 12:30 and takes off to a new destination at 14:50. It is obvious that gate G cannot be assigned to any other aircraft during that time period. But if it can be assigned to an aircraft that lands after 14:50.

6 Find a large file with many departures using this same format in ATENEA, week 4.

Therefore, the assignment of doors is a dynamic process so that the situation changes throughout the day. Version 4 of your application should consider this complexity. To that purpose you will need the following functions.

```
def AssignGatesAtTime (bcn, aircrafts, time)
''' This function receives an object of class BarcelonaAP, bcn, a list of aircrafts arriving and/or taking-off during the day, and a string with an exact hour the day (01:00, 02:00,…, 23:00). The function must update the state of bcn by assigning gates to the aircrafts in the list that lands during the one-hour long period starting in the received time hour. To do that, the function must first free the gates with aircrafts already departed. Then, loop through the list of aircrafts and assign gates to those that will land during the period under consideration. The function must return the number of aircrafts landing in the considered period that were not assigned due to full gate occupancy.
'''

def PlotDayOccupancy (bcn, aircrafts)
''' This function receives an object of class BarcelonaAP, bcn, a list of aircrafts arriving and/or taking-off during the day. The state of bcn corresponds to the start of the day (that is, only night aircrafts). The function must build a plot showing the total number of gates assigned in every terminal in every hour period of the day, and also the number of aircraft that were not assigned in every hour period.
'''
```

Finally, you must extend the user interface with the elements that should allow the user to use the new functionalities. Specially interesting would be the possibility to show the gate occupancy as suggested for version 3, but now for every hour period, on demand of the user (this would be considered as a nice extra functionality).

Remember that version 4 is the time to impress your teachers with your creativity by incorporating the most interesting and spectacular functionalities into the project, depending on the motivation and desire to work that you have left.

Quality and grading criteria for version 4

Correct (4 points) • All required files have been developed • The test files allow you to verify the correct functioning of all the functions associated with the classes. • Through the graphical interface the user can: o ALL FUNCTIONALITIES FROM PREVIOUS VERSIONS o Load departures and merge them with arrivals o Assign gates in every hour period of the day o Plot the day occupancy in every terminal and every hour period
Extra functionalities (2 points) • The extra features are spectacular • The extra features are complex
Presentation (1 point) • The video makes it clear which functionalities are operational among those planned for this version. • The video is spectacular and fun.

Robust (1 point) • The application resists without blocking the errors that the user may make when interacting with it. It never crashes. • The application detects and reports errors when reading files.
Friendly (1 point) • The user has no doubt when interacting with the graphical interface. • The graphical interface is visually attractive.
Organized and documented (1 point) • The code is well structured and indented. It is easy to locate where in the code every functionality is implemented. • Clarifying comments have been added.

Annex 1

Showing data in GoogleEarth

In order to show points and routes in GoogleEarth you must prepare files with information in Keyhole Markup Language (KML) format, which is a notation for expressing geographic information used by many map applications.

KML files are just text files, so that you can open them using any text editor. For instance, you can use PyCharm to open a see a KLM file. Additionally, KML can be opened with a map application, such as GoogleEarth, just double-clicking on them. Then the KML content is interpreted and geographical maps are shown on the screen.

In the example below you can see the structure of 2 simple KML files and how they are visualized when you double-click the file from your operating systems. Of course, you need to have GoogleEarth installed to be able to show such images.

From the examples of KML given below, observe how these files use a tag-based structure with nested elements and attributes. Tags are words such as Document, Point,Name, etc. inserted between '<' and '>' characters. Every tag has a corresponding closing tag. For instance, <name> opens the name section, while </name> closes it. Sometimes sections can have other sections inside, such as Placemark, that in the example contains the sections Name, Description and Point.

The first KML example, PointExample.kml, shows how to place a Point in a map. The second KML, LineExample.kml, shows how to draw a line given by a sequence given by 2 points.

Observe that there is an initial part and end part of the text file that is always fixed. Then, to add more points or more lines you just need to add new text lines with new Placemarks, Points, LineStrings and /or Coordinates.

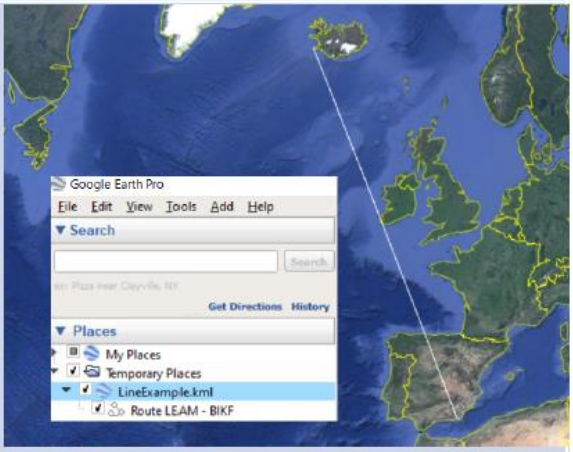
Before starting to program, we suggest you download both files from Atenea. Try to open first with the text editor, and then with GoogleEarth. Remember you need to install GoogleEarth in your computer (from http://www.google.com/earth/download/ge/) to see the maps. But for this project it is enough if you can write correct KML files, which can be checked with the text editor.

EXAMPLE 2 Points

```
<kml xmlns="http://www.opengis.net/kml/2.2">
<Document>
  <Placemark> <name>LEBL</name>
    <Point>
     <coordinates>
       2.07833,41.29694
     </coordinates>
    </Point>
  </Placemark>
  <Placemark> <name>LEMD</name>
    <Point>
     <coordinates>
       -3.700346,40.41669
     </coordinates>
    </Point>
  </Placemark>
</Document>
</kml>
```

EXAMPLE 1 Line

```
<kml xmlns="http://www.opengis.net/kml/2.2">
<Document>
 <Placemark>
  <name>Route LEAM - BIKF</name>
  <LineString>
   <altitudeMode>clampToGround</altitudeMode>
   <extrude>1</extrude>
   <tessellate>1</tessellate>
    <coordinates>
      -2.3733335667,36.8422863167
      -22.6144444444,63.9869444444
    </coordinates>
  </LineString>
 </Placemark>
</Document>
</kml>
```



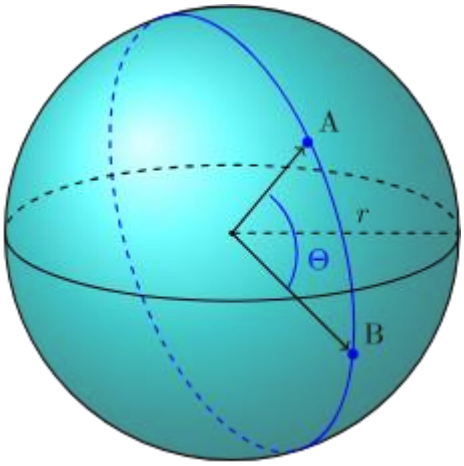
Although you don't need to, if you are curious about KML, you can read more details about its properties in https://developers.google.com/kml/documentation/kml_tut .

NOTE: Python also has libraries (i.e. simplekml or pykml) that helps a programmer to write complex layers on top of a map. You are free to use them, but in the INFO1 project, it is easier if you write text files directly, using your knowledge from BLO6.

Annex 2

Haversine formula

Given 2 points on the Earth surface, A and B, with latitudes represented by $\varphi 1$ and $\varphi 2$ and their longitudes by $\lambda 1$ and $\lambda 2$ respectively, the Haversine formula can be used to calculate an approximation of the great-circle distance between the two points – that is, the shortest distance over the earth's surface.



A is at ($\phi 1$, $\lambda 1$)
B is at ($\phi 1$, $\lambda 2$)

Haversine distance is $d = r \cdot c$

where

r is earth's radius (mean radius = 6371km)

$c = 2 \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{(1-a)})$

and

$a = \sin^2(\Delta\varphi/2) + \cos \varphi 1 \cdot \cos \varphi 2 \cdot \sin^2(\Delta\lambda/2)$

$\Delta\varphi = |\varphi 1 - \varphi 2|$

$\Delta\lambda = |\lambda 1 - \lambda 2|$

IMPORTANT NOTE: all angles need to be converted to radians !

Once you have program your function you can check here if you have done correctly the function.